ADT Stack • container in which access is restricted to the TOP. (one end of the container) (LIFO)

→ DynamicArray (DS): ↓ NO ITERATOR!

Stack:
- capacity: Int
- nrElem: Int
- elems: TElem[]

Array-Based: TOP at the end
SLL-Based: TOP at the head
DLL-Based: TOP at either head / tail

- init(S) → Θ(1)!
- destroy(S)
- push(S, e) → BC: Θ(1); WC: Θ(n) resize ⇒ Θ(1) am.
- pop(*) → Θ(1) → return.
- top(*) → Θ(1) remove + return
- isEmpty(S) → Θ(1)
- size() → Θ(1)

ADT PriorityQueue • container in which access is restricted to FRONT, REAR and each elem. has an associated priority (TPriority), we can access ONLY ELEM with THE HIGHEST PRIORITY! (HPF)

↓ NO ITERATOR!

→ DynArray:

TElemPQ
- elem: TElem
- priority: TPriority

PriorityQueue:
- capacity: Int
- nrElem: Int
- elems: TElemPQ[]
- relation: TRelation

- init(pq, R) ✓
- destroy(pq) ✓
- push(pq)
- pop(pq)
- top(pq)
- isEmpty(pq)

ADT Queue • container in which access is restricted to the FRONT and REAR.

(*): for circular array ∃ a conv: front = position of first element, rear = position after the last element (front increments) ⇒ push, pop, front in Θ(1)

pop ↑ (FIFO) ↑ push

→ CircularArray: (convention) (*)

Queue: nrElem: Int
- capacity: Int
- elems: TElem[]
- front: Int
- rear: Int

+ isFull()!

→ StaticArray:

Queue:
- capacity: Int
- nrElems: Int
- elems: TElem[]
- front: Int
- rear: Int

+ isFull() needed!

→ DynamicArray same repres, but we can resize()

- init(q) → Θ(1)
- destroy(q)
- push(q, e) → BC: Θ(1); WC: Θ(n) ⇒ Θ(1) am.
- pop(q) → Θ(1)
- top(q) → front of q → Θ(1)
- isEmpty(s) → Θ(1)
- front(q) = top(q) (DLL does not care)

push = insert at rear, rear increments
pop = remove from front, front increments

where to place FRONT and REAR w an ARRAY? ⇒ good if we push more

SLL: front (ind=0) rear

DLL: rear (ind=0) front ⇒ good if we pop more

LINKED LIST (DS) • linear DS in which the order of elems is det. not by indexes, but by a pointer which is placed in each elem.

• NODE = data + pointer to the address of next node (+/- prev)

SLL:
- SLLNode:
  - info: TElem
  - next: ↑SLLNode
- SLL:
  - head: ↑SLLNode

DLL:
- DLLNode:
  - info: TElem
  - next: ↑DLLNode
  - prev: ↑DLLNode
- DLL:
  - head: ↑DLLNode
  - tail: ↑DLLNode

SSLL (sorted SLL)
- SSLLNode:
  - info: TComp
  - next: ↑SSLLNode
- SSLL
  - head: ↑SSLLNode
  - rel: ↑relation
- init, insert, search (stop earlier), delete, getFromPos, iterator → like SLL

• search, add to ≠ places, delete from ≠ places, get from pos
• search(sll, elem) → O(n) (pay attention what you return)
• insertFirst(sll, elem) → Θ(1)
• insertAfter(sll, current, elem) → Θ(1)
• insertPos(sll, pos, elem) → O(n)
...

• same op. as SLL
• insertLast(dll, elem)
• insertPos(...)

SLLIterator
- list: SLL
- currentElem: ↑SLLNode

DLLIterator
- list: DLL
- currentElem: ↑DLLNode

• init(it, sll)

ADT Deque • we can insert + delete from both ends.

Deque (CircularArray)
- cap: Int
- nrElem: Int
- elems: TElem[]
- front: Int
- rear: Int

- ~~same op. as~~
- init(), destroy(), pushFront(e), → Θ(1) am. pushBack(e), → Θ(1) am. popFront(), isEmpty() popBack(), front(), back()

SLLA (no pointers)

• allows to add/insert anywhere as long as links are set correctly.

SLLA: (two arrays)
- elems: TElem[] (elems)
- next: Integer[] (links)
- cap: Integer (same)
- head: Integer (index)
- firstEmpty: Integer (index)

• same op. as SLL.

SLLAIterator: → one array for elems, one for next
- slla: SLLA
- current: Integer

DLLA (no pointers)

DLLANode:
- info: TElem
- next: Integer
- prev: Integer

DLLAIterator
- list: DLLA
- currElem: Integer

DLLA (one array)
- nodes: DLLANode[]
- cap: Integer
- head: Integer
- tail: Integer
- firstEmpty: Integer
- size: Integer

• allocate(), free() to make it closer to a LL w Dyn. Alloc.

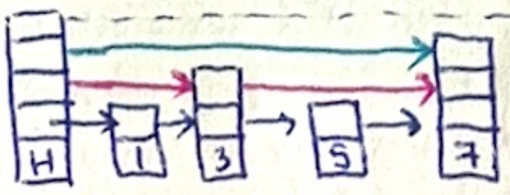
SLCircularList • each node has a successor "last node" = the node whose next is the head of the list • retain last node to help insertion

CSLLNode:
- info: TElem
- next: ↑CSLLNode

CSLL:
- head: ↑CSLLNode

• insertFirst Θ(n)
• deleteLast → Θ(n)
• iterator → invalid when the next of currentElem is head. ⊕ boolean flag

SkipList (DS) • fast SEARCH in an ordered LL.



Node:
- info: TElem
- next[0..lev]: Node

Skiplist:
- head: Node
- maxLev: Int
- rel: Relation

• search: start from top level, move right while next < elem, if cannot move right, go down one lev. ⇒ WC: O(n) ⊖ extra space for links

SpecialStack ⇒ AUX STACK

• getMin → Θ(1) (minstack)
• other op → Θ(1)

SpecialStack:
- elementStack: Stack
- minStack: Stack

ADTDeque

→ dynamic array of fixed-size arrays

PRINT ANY CONTAINER:

```
subalgorithm print(c) is:
    iterator(c, it)
    while valid(it) execute
        elem ← getCurrent(it)
        print elem
        next(it)
```

(1)

ADTPriorityQueue

→ Dyn.Array / LL → decide how we store elems in the array/list:
I. ORDERED BY PRIORITY or
II. ORDERED BY INSERTION

XOR LinkedList

• link = prev XOR next

XORNode:
- info: TElem
- link: ~~Address~~ ↑Int

XORList:
- head: XORNode
- tail: XORNode

ARRAY (DS)

- fixed nr. of elems
- elems of the same type
- contiguous memory block
- address of the first elem.

⊕ elems accessed in $\Theta(1)$ ✓

⊖ we don't always know from the start the nr. of elem.

DYNAMIC ARRAY

→ traverse either by for i=1...nrElem or by ArrayIterator

```
capacity: Integer
nrElem: Integer
elems: TElem[]
```

- size() → Θ(1)
- getElem(p) → Θ(1)
- setElem(p,e) → Θ(1)
- iterator() → Θ(1)
- addToEnd(e) → Θ(1) am.
- addToPos(p,e) → O(n)
- deleteFromEnd() → Θ(1)
- deleteFromPos(p) → O(n)
- deleteGiven(e) → Θ(n)

ArrayIterator (not needed, we have positions!)

```
array: DynamicArray
current: Integer
```

- first() → Θ(1)
- next() → Θ(1)
- valid() → Θ(1)
- getCurrent() → Θ(1)

STATIC ARRAY

```
Array:
    elems[CAPACITY]
```

(ADT) ↓ correct DS, but sometimes treated like ADT because its operations are discussed directly.

⊕ add, remove in Θ(1) time at THE END

⊖ add, remove in Θ(n) time at the BEGINNING. ⇒ LL. (better)

INSERTION:
- DA → optimised search ($O(\log_2 n)$), but insertion O(n)
- LL → optimal insertion, but search is O(n) (Θ(1) insertion)

- init(da, cap)
- resize(da)
- destroy(da)

ADT List • only container with POSITIONS • ∃ order, ∃ pos.

→ Dyn. Array:

```
List:
    cap: Int
    nrElem: Int
    elems: TElem[]
```

Iterator

```
TPosition = Int (index)
```

- init(l) • search(l,e) • isEmpty(l)
- first(l), last(l) • size(l)
- valid(l,p) • addToB(l,e)
- next(l,p) • addToE(l,e)
- prev(l,p) • addBefP(l,p,e)
- getElem(l,p) • addAftP(l,p,e)
- position(l,e) • remove(l,p)
- setElem(l,p,e) • remove(l,e)

(1,A), (1,B), (1,A), (1,C)

MultiMap (R1) (*)

MultiMap (R2):

(1, {A,B,C}) = elems[0]

```
TElem:
    Key: TKey
    VL: ValuesList
MultiMap: (R2)
    capacity: Int
    nrElem: Int
    elems: TElem[]
```

ADT IndexedList • list in which TPosition = Int

⇒ ∄ first, last, next, prev, valid.

- addToPos(l,i,e)

ADT IteratedList • list in which TPosition = Iterator ⇒ ∄ next, prev, valid

- addToPos(l,it,e)

ADTSortedList • one add op., ∄ setElem() Indexed/Iterated

ADT Map (dictionary) • no positions, no order, unique keys, store (key, value) pairs, values may not be unique

(*) Map: → add checks if ~~elem~~ key already exists (key)

```
TElem: (R1)
    key: TKey
    value: TValue
```

- init(m)
- destroy(m)
- add(m,k,v) "PUT"
- remove(m,k)
- search(m,k)
- iterator(m,it)
- size(m)
- isEmpty(m)
- keys(m,s) SET
- values(m,b) BAG
- pairs(m,s) SET

```
Map:
    capacity: Integer
    nrElem: Integer
    elems: TElem[]
```

ADT SortedMap • Map + order given by a relation which is passed to the init() fct.

```
SortedMap:
    capacity: Int
    nrElem: Int
    elems: TElem[]
    relation: Relation
```

- add preserves rel ℜ
- iterator returns in sorted order.

ADT MultiMap • a key may have more values (unique values or not); 1→A; 1→A; 1→A is valid ✓ 1→A; 1→B; 1→C is valid ✓ (add does not check key uniqueness)

ADTSortedMultiMap

- init, iterator
- add checks rel ℜ.

Matrix with compressed sparse column repres:

obtained from Cols ↑

```
Matrix:
    Lines: Integer[]
    Cols: Integer[]
    Values: TElem[]
    capacity: Integer
    nrElem: Integer
    nrLines: Integer
    nrCols: Integer
```

→ Values[i] = (Lines[i], column)

→ Cols[j] = pos in Values where column j starts.

- same operations as Matrix

Matrix with compressed sparse line representation:

→ Values[i] = (line, Cols[i])

→ Lines[i] = pos in Values where row i starts.

ADT Bag: • no positions, no order, no uniqueness

→ Dynamic Array (DS):

```
Bag: R1 → more memory
    capacity: Integer
    nrElem: Integer
    elems: TElem[]
```

✓ • add(b,e) → Θ(1) am (!)
✓ • remove(b,e) → Θ(n)
• size(b) → Θ(1)
✓ • search(b,e) → Θ(n)
• init(b)
• destroy(b)
✓ • iterator(b,it)
✓ • nrOccurrences(b,e) → Θ(n)
• isEmpty() → Θ(1)

```
Bag: R2 → less memory
    capacity: Integer
    nrElem: Integer
    elems: TElem[]
    freq: TElem[]
```

- nrOccurrences (easier)
- iterator (harder)

```
Bag: R3
    capacityElems: Integer
    nrElems: Integer
    elems: TElem[]
    capacityPos: Integer
    nrPos: Integer
    positions: TElem[]
```

```
Bag: R4
    capacity: Integer
    nrElem: Integer
    elems: TElem[]
```

② Bag: R3

No positions ⇒ more performance

Iterators ⇒ faster even on Lists (where we already have pos)

ADT SortedBag

- Bag + relation passed to the init() function

```
SortedBag: R1
    capacity: Int
    nrElem: Int
    elems: TElem[]
    Relation ℜ
```

- add must preserve sorting by ℜ
- iterator returns in sorted order by ℜ
- init(b,ℜ)

SortedBag: R2

⇒ ∄ remove elem. from a position

ADT Set • no positions, no order, unique.

→ DynamicArray (DS):

```
Set:
    capacity: Int
    nrElem: Int
    elems: TElem[]
```

- init(s)
- add(s,e)
- remove(s,e)
- search(s,e)
- size(s)
- isEmpty()
- iterator(s,it)
- destroy(s)

- Bag + add(e) first checks search(e) and inserts only if elem. e is not already there

ADT SortedSet • Set + relation passed to the init() function

- add must preserve sorting by ℜ
- iterator returns in sorted order

```
SortedSet:
    capacity: Int
    nrElem: Int
    elems: TElem[]
    Relation ℜ
```

ADT Matrix • unique positions determined by line, col.

= TWO-DIMENSIONAL ARRAY.

```
Matrix: → more memory for sparse matrices
    nrLines: Int
    nrCols: Int
    elems: TElem[]
```

- init(mat, nrLines, nrCols)
- nrLines(mat) → Θ(1)
- nrCols(mat) → Θ(1)
- element(mat,i,j) → Θ(1)
- modify(mat,i,j,val) → Θ(1)

Access formula:

pos = i * nrCol + j

```
SparseMatrix: (many 0)
    Lines: Integers[]
    Cols: Integers[]
    Values: TElem[]
    capacity: Integer
    nrElem: Integer
    nrLines: Integer
    nrCols: Integer
```

Values[i] = (Lines[i], Cols[i])

- init, nrLines, nrCols, element, modify

Iterators → useful for iterating through the elements of any container in an uniform way

ADT Priority Queue (emergency room)

- each elem has a priority (TPriority)
- only elem with highest priority is accessible • FIFO
- HPF
- relation R
- operations:
  1) init (pq, R)
  2) destroy (pq)
  3) push (pq, e, p)
  4) pop (pq)
  5) top (pq)
  6) isEmpty (pq)
- no iterator!

K-ary tree:

~~info~~

Node:
- info: TElem
- parent: ↑Node
- leftChild: ↑Node
- rightSibling: ↑Node

Traversals:
1) DFS (depth)
2) BFS (level-or)

DFS → stack, visited
BFS → queue, visited

BinaryHeap (DS) → efficient repres. for PriorityQueues = dyn. array + binary tree
=> DA = levels
→ each node has 2 children except for the last 2 rows (left → right)
→ elem. on pos i => children are on pos 2i, 2i+1; => parent of i is [i/2]; STRUCTURE = no gaps; PROP (max / min)

HT not suitable for SORTED stuff, except the SortedMap; HT => FAST SEARCH

ORDERED TREE: order of children is relevant

COMPLETE TREE: height = $\log_2 n$

HEIGHT OF A TREE (rec.) (to the root)
height(node) O(n)
if node == null
  return -1
return 1 + max(height(node.left), height(node.right))

node ↑ leaf

degree = nr. children
leaf / internal n.
DEPTH: root
HEIGHT: ↓ node
longest

Linear probing: $h_1(k,i) = (h'(k) + i) \% m$
Double hashing: $h_2(k,i) = (h'(k) + i \cdot h''(k)) \% m$
$h'(k) = k \% m$ or other method (given); $h''(k)$;

TREE → conn., acyclic graph; One node = ROOT;

BINARY TREE
- ordered tree with max 2 children (left / right child)
- FULL when every internal node has 2 ch.
- COMPLETE when all leaves on the same level

BINOMIAL TREE
- ~~order k → 2^k nodes~~
- order k => 2^k nodes => k children on the level after the root

ADT BinaryTree ~ BST

Node:
- info: TElem
- left: TNode
- right: TNode

BT:
- root: TNode

(linked repres. and dynamic allocation) !

op: init(s), push(s,e), pop(s,e), isEmpty(s), top(s) → stack / queue operations

BINARY SEARCH TREE
- BT + extra property: each elem left subtree ≤ current ≤ each right subtree.
- FOR SORTED CONTAINERS
- O(height)
- insert: O(n)

ADT BinaryTree (DLLA)

Node:
- info: TElem
- left: Int
- right: Int

BT:
- nodes: ↑Node[]
- root: Int
- firstEmpty: Int
- cap: Int

TreeNode: info: TElem; parent: ↑TreeNode; child: ↑TreeNode; sibling: ↑TreeNode
Tree: not ordered

BINARY HEAP
- single + complete BT.
- every level is filled, except maybe the last level.
- it merges inefficiently
- a BT having a heap structure and a heap property (min / max)
- O(log₂ n) ADD
- Θ(1) get ROOT
- O(log₂ n) DELETE (ROOT)

BINOMIAL HEAP
- collection of binomial trees → at most $\log_2 n$
- height = order k
- delete root => k binom. t.
- two trees with the same order k can be merged into a binomial tree of order k+1 by setting one to be the LEFTMOST child of the other
- sorted LL

MERGE

ADT Stack

- the access to the elements is restricted to one end of the container (TOP of the stack)
- LIFO
- For DynamicArrays, you should put the top of the stack towards the end of the array!
  → pos 1 → push/pop in Θ(n)
  → end pos → push/pop in Θ(1)
- ARRAYS – static / dynamic
- LINKED LISTS – SLL / DLL

* Special stack with getMinimum operation with Θ(1) time?
→ use 2 stacks.

BinaryHeap
- cap: Integer
- len: Integer
- elems: TElem[]

(rel: TRelation) → For PriorityQueue

- the only operations on a heap:
  - push
  - pop

HeightOfHeap = $\log_2 n$ levels =
= 1 + 2 + 4 + 8 + ... = n

- bubble-up → swap with parent until reached the correct position (prop.)
- bubble-down → swap with best child until leaf / respects prop.
- leaves = nodes with indices: [n/2]+1, ..., n.; add to heap = push to PriorityQueue; remove from heap = pop. → return root ≠ top

BUILD BIN. TREE from arithmetic expression:
- operand (a) → push to stack; • operator => pop 2 from stack => merge those trees (postfix as input)
- last thing popped goes on the RIGHT!

BinaryNode
- info: TElem
- parent: BinaryNode
- leftChild: BinaryNode
- ~~rightChild~~:
- rightSibling: BinaryNode

BinomialTree
- root: BinomialNode

- merge 2 heaps → O($\log_2 n$)
- push (= create a bin. heap with only that node) → O($\log_2 n$) WC; Θ(1) amortized.
- top (min / max elem) = root (elem with highest priority) => O($\log_2 n$)
- pop (rem. min) O($\log_2 n$)

PriorityQueue Operation Compl.

Op	Sorted	Non-Sort	Heap
push	O(n)	Θ(1)	O($\log_2 n$)
pop	Θ(1)	Θ(n)	O($\log_2 n$)
top	Θ(1)	Θ(n)	Θ(1)

```
Subalgorithm buildTree (postfix)
    init(S)
    for e in postfix
        if e is an operand
            node ← allocate()
            [node].info ← e
            [node].left ← NIL
            [node].right ← NIL
            push(S, node)
        else
            e1 ← pop(S); e2 ← pop(S)
            node ← allocate()
            [node].info ← e
            [node].left ← e2
            [node].right ← e1
            push(S, node)
    init(bt)
    bt.root ← pop(S)
    buildTree ← bt
```

AVL TREE ~ BST
- BT + extra prop: height(left) − h(right) ∈ {−1, 0, 1} (OST)

AVLNode:
- info: TComp
- left: ↑AVLNode
- right: ↑AVLNode
- h: Integer

BinHeap
- capacity: Int
- nrElem: Int
- elems: TElem[]
- insert: O($\log_2 n$)

AVLTree
- root: ↑AVLNode

(↑ relation)
→ sorting alg.
asc. order => build a min. heap and remove elems one by one
O(n log₂ n)

BINOMIAL HEAP ~2 ~priority queue
- a collection of binomial trees
- it merges two heaps efficiently → at most $\log_2 n$ Binom. Tr.
- order k => 2^k nodes; n = 14 = 1110 = $2^1 + 2^2 + 2^3$ => BT of orders 1, 2, 3.
- at most one binomial tree of each degree (BFS)

TOP view → columns → first node found; queue; map
ANCESTORS of degree k => O(min(n, 2^k)) = PRINT LEVEL k
HEIGHT of a tree: using DFS traversal
ROOT HAS DEGREE 0.

DT = Domain (Val.) + Interface (op) ← set of values (dom.) with a set of op. DSA reference sheet

ADT = no implementation = DT with properties

CONTAINER = collection of data, ± types. which allows accessing elements through an iterator.

SUBALGORITHM = subprogram that does not return a value.

FUNCTION = subprogram that returns a value.

AMORTIZED = total cost / nr. operations.
(ex: addToEnd in DArray is $\Theta(1)$ amortized because $\Theta(n)$ rarely happens, when resizing occurs.

COMPLEXITY

abstraction = separation of domain and interface from implementation

- Complexity = how do the nr. of steps change if n increases/...
- Nr. of steps = depends on SIZE. ($T(n)$).
- O - notation = upper bound
  - ex: $T(n) = n^2 + 2n + 2$
    - $\Rightarrow O(n^2), O(n^3)$ ✓
- Ω - notation = lower bound
- Θ - notation (exact)
  - ↳ AC, BC, WC. (first occurence of a nr. in an array)
    - ↓ min. ↓ max. nr. steps

$$\sum_{I \in D} P(I) \cdot E(I) = pb * nr\_op$$

- BC = WC = AC $\Rightarrow$ TC = Θ
- BC ≠ WC ≠ AC $\Rightarrow$ TC = WC $\Rightarrow$ O

BinaryTree w linked repres (BST, AVL, BT)

Node: info: Integer, left: TNode, right: TNode

BT: root: TNode

- root(bt) • left(bt) • right(bt) • isEmpty(bt)
- init(bt) • destroy(bt) • initTree(bt, left, e, right)
- initLeaf(bt, e) • insertLeftSubtree(bt, left) • insertRightSubtree(bt, right)
- iterator(bt, bar, i)

BinaryTreeIterator

bt: BinaryTree
stack: Stack<Node> or Queue<Node>
current: Node

STRUCT (static DS) = RECORD

static ARRAY (DS)

- orders matters (contiguous mem.)
- fixed nr. of elements (capacity)
- same type of elements (type)
- access in $\Theta(1)$ (+)
- static (−)

DynamicArray

cap: Integer
nrElem: Integer
elems: TElem[]

BinaryTree w array repres.

BT:
cap: Integer
~~root: Integer~~
elems: TElem[]

elems[1] = root

ADT Bidirectional map

BDHNode

K: TKey
v: TValue
nextKey ↑BDHNode
prevKey ↑BDHNode
nextValue ↑BDHNode
prevValue ↑BDHNode

BDHMap

m: Integer
keyTable(↑BDHNode)
valueTable(↑BDHNode)[]
hKey: ↑Function
hValue: ↑Function
size: Integer

- init(bdm) → $\Theta(1)$
- insert(bdm, k, v) → $\Theta(1)$, amortized
- search(bdm, k) → $\Theta(1)$, avg
- reverseSearch(bdm, v) → $\Theta(1)$
- remove(bdm, k) → $\Theta(1)$

ADT BAG

(shopping cart)

- order does not matter
- uniqueness does not matter
- no positions
- operations:
  1) init(b)
  2) add(b, e)
  3) remove(b, e)
  4) search(b, e)
  5) size(b)
  6) nrOccurences(b, e)
  7) destroy(b)
  8) iterator(b, i)
     - ↳ init(i, b)
     - ↳ valid(i)
     - ↳ first(i)
     - ↳ next(i)
     - ↳ getCurrent(i, e)
- representations:
  - ✓ 1) LIST (dynamic array)
  - 2) 2 LISTS (elem + freq.)
    - or
    - 1 LIST of pairs

PYTHON IMPLEMENTATION (1)

```
class Bag:
    def __init__(self):
        self.__bag = list();

    def add(self, e):
        self.__bag.append(e);

    def remove(self, e):
        if e in self.__bag:
            self.__bag.remove(e);
            return True
        return False

    def search(self, e):
        return e in self.__bag

    def size(self):
        return len(self.__bag)

    def nrOfOccurences(self, e):
        count = 0
        for elem in self.__bag:
            if elem == e:
                count = count + 1
        return count

    def iterator(self):
        return BagIterator(self)
```

```
class BagIterator:
    def __init__(self, b):
        self.__bag = b
        self.__current = 0

    def valid(self):
        return self.__current < self.__bag.size()

    def first(self):
        self.__current = 0

    def next(self):
        self.__current += 1

    def getCurrent(self):
        if (self.valid()):
            return self.__bag.__Bag__bag[self.__current]
        raise ValueError()
```

ADT SortedMultiMap

Representation:

TElem: K: TKey, vl: List

SMMNode: info: TElem, next: ↑SMMNode

Operations:
- init
- valid
- next
- getCurrent

singly linked repres. with dynamic alloc.

SMMIterator: cNode: ↑SMMNode, cValue: ListIterator, smm: SMM

SMM: head: ↑SMMNode, size: Integer, rel: Relation

ADT Sequence

- operations: $\Theta(1)$
  1) isEmpty(sequence)
  2) init(sequence)
  3) removeFirst(sequence)
  4) destroy(sequence)
  5) addToEnd(sequence, (K, v))

BDHMap

BDHNode: K: TKey, v: TValue, next_key, prevKey, nextValue, prevValue: ↑BDHNode

BDH: keyTable, valueTable(↑BDHNode)[], m: Int, h-key, h-value: ↑Function, size: Int

ADT SortedMap

Operations:

Representation:

Node: K: TKey, next: Node, next_sorted: ↑Node, prev_sorted: ↑Node

SM: m: Integer, T: (TNode)[], K: TFunction, rel: Relation

```
subalgorithm insertFirst(slla, elem) is:
  if slla.firstEmpty = -1 then
    newElems ← @ array ... *2 pos
    newNext ← @ array ... *2 pos
    for i←1, slla.cap execute
      newElems[i] ← slla.elems[i]
      newNext[i] ← slla.next[i]
    for i←slla.cap+1, slla.cap*2-1 ex
      newNext[i] ← i+1
    newNext[slla.cap*2] ← -1
    //free slla.next, slla.elems (opt.)
    slla.elems ← newElems
    slla.next ← newNext
    slla.firstEmpty ← slla.cap + 1
    slla.cap ← slla.cap * 2
  newPosition ← slla.firstEmpty
  slla.elems[newPosition] ← elem
  slla.firstEmpty ← slla.next[slla.firstEmpty]
  slla.next[newPosition] ← slla.head
  slla.head ← newPosition
```

Θ(1) amortized

```
subalg deleteElem(slla, e):
  prev ← -1
  current ← slla.head
  while current ≠ -1 and
    slla.elems[current] ≠ e execute
      prev ← current
      current ← slla.next[current]
  if current ≠ -1 then
    deleteElem ← false
  else
    if prev = -1 then
      slla.head ← slla.next[slla.head]
    else
      slla.next[prev] ← slla.next[
                          current]
    slla.next[current] ← slla.firstEmpty
    slla.firstEmpty ← current
    deleteElem ← true
```

```
subalgorithm insertPos(dlla, e, pos):
  if pos<1 or pos > dlla.size+1 then
    @throw exception
  occupiedPos ← allocate(dlla);
  if occupiedPos = -1 then
    newArray ← @ new array(*2)
    for i←1 to dlla.cap execute
      newArray[i] ← dlla.nodes[i]
    for i←dlla.cap+1 to dlla.cap*2 ex
      newArray[i].next ← i+1
      newArray[i].prev ← i-1
    newArray[dlla.cap*2].next ← -1
    newArray[dlla.cap+1].prev ← -1
    dlla.firstEmpty ← dlla.cap+1
    dlla.cap ← dlla.cap*2
    dlla.nodes ← newArray
    occupiedPos ← allocate(dlla)
  dlla.nodes[occupiedPos].info ← e
  if pos=1 then // insert at beginning
    dlla.nodes[occupiedPos].next ← dlla.head
    if dlla.head = -1 then // empty list
      dlla.head ← occupiedPos
      dlla.tail ← occupiedPos
    else // normal list
      dlla.nodes[dlla.head].prev ← occupiedPos
      dlla.head ← occupiedPos
  else if pos = dlla.size+1 then // insert at the end
    dlla.nodes[occupiedPos].prev ← dlla.tail
    dlla.nodes[dlla.tail].next ← occupiedPos
    dlla.tail ← occupiedPos
  else // normal insertion (middle)
    currentN ← dlla.head
    currentP ← 1
    while currentP < pos-1 (BEFORE)
      currentN ← dlla.nodes[currentN].next
      currentP ← currentP+1
    dlla.nodes[occupiedPos].prev ← currentN
    dlla.nodes[occupiedPos].next ← dlla.nodes[currentN].next
    dlla.nodes[dlla.nodes[currentN].next].prev ← occupiedPos
    dlla.nodes[currentN].next ← occupiedPos
  dlla.size ← dlla.size+1
```

HASH TABLES (DAccessTable generalis.)

- for DAT → Θ(1), but 2xB and memory ↑ss. (faster, but more memory) → T[K]
- HT → slightly slower, but less memory
  ↳ T[h(K)]
- hash functions should min. collisions

I SEPARATE CHAINING (LL on a slot)

Node: (α>1)] HT:
key: TKey can! T: ↑Node[]
next: ↑Node m: Integer
h: TFunction

- search: O(α+1)
- insert: Θ(1) ± search ...

Θ(1) = time needed to compute hash fct.
α = average time needed to search one of the m lists, WC: Θ(m) when all are in a SLL

HTIterator
ht: HT
currentPos: Int
currentNode: ↑Node

II COALESCED CHAINING (in slot, has next)

HT: (α≤1)
T: TKey[]
next: Integer[]
m: Integer
firstEmpty: Integer
h: TFunction

- insert: Θ(1)AC, Θ(n)WC
- remove Θ(m), AC: Θ(1)
- search: BC: Θ(1), AC: Θ(α+1), WC: Θ(n)

HTIterator:
ht: HT
current: Integer

III OPEN ADDRESSING (α≤1)

(no pointer, no next)
- probe sequence

Linear probing: $h(k,i) = (h'(k)+i) \% m$
Quadratic probing: $h(k,i) = (h'(k) + c_1 \cdot i, c_2 \cdot i^2) \% m$
Double hashing: $h(k,i) = (h'(k) + i \cdot h''(k)) \% m$

HT:
T: TKey[]
m: Integer
h: TFunction

α = ct ⇒ Θ(1)
WC: Θ(n)

HASH TABLES DO NOT MAINTAIN POSITIONS / SORTED RELATIONS
⇒ SET, BAG, MAP, MULTIMAP can be represented using HT.
(order does not matter in these containers) Θ(1) usually (insert + search)

Cuckoo hashing T1, T2; send one to another.

REMOVE = the only operation on HT that cannot be efficiently implemented (searching is expensive) → O(n)
(with open addressing)
(sep. chaining is fine)

```
subalgorithm printContainer(c) is:
  iterator(c, it)
  while valid(it) execute
    elem ← getCurrent(it)
    print elem
    next(it)
```

SAME FOR ALL CONTAINERS!

LINKED HASH TABLE (DS) • predictable iteration order (= order of insertion)

Node:
info: TKey
nextH: ↑Node
nextL: ↑Node
prevL: ↑Node

LinkedHT: (for efficient removal)
m: Integer
T: (↑Node)[]
h: TFunction
tail: ↑Node, head: ↑Node

(if you need in sorted order, store Relation)

Perfect hashing → multiple hash functions ⇒ use universal hashing
$\mathcal{H} = \{H_{a,b}(x) = ((a*x+b) \% p) \% m\}$ ⇒ hash table of hash tables

14/06 = 14

XOR Linked List

- memory-efficient
- DLL where every node retains one link (XOR between the previous and next node)

XORNode:
- info: TElem
- link: ↑Integer //XOR

XORList:
- head: ↑XORNode
- tail: ↑XORNode

Traversal:

subalgorithm . .

SLL

current ← sll.head
current ← [current].next

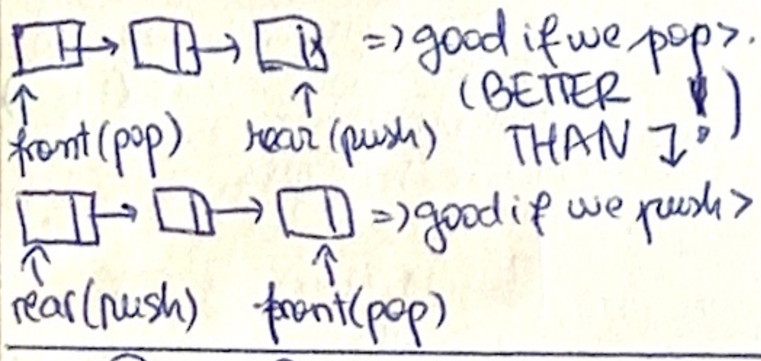
- top of the stack should be at the beginning of the list

Queue on SLL

Node:
- info: TElem
- next: ↑Node

Queue:
- head: Node
- tail: Node
- nrElem: Int

- Store tail because otherwise push would require traversing the whole list.



PriorityQueue on DLL

PQElem:	Node	PrQueue:
elem: TElem	info: PQElem	head: Node
priority: TPriority	next: Node	rel: QRel
		nrEl: Int

- does not matter where the top of the stack is (in the list)

Queue on DLL

Node:
- info: TElem
- prev: Node
- next: Node

Queue
- head: Node
- tail: Node
- nrElem: Int

- does not matter where we put front / rear
- does not improve Queue op.

ADTDeque on DLL

Node:
- info: TElem
- prev: Node
- next: Node

Deque
- head: Node
- tail: Node
- nrElem: Int

Skip List

- store sorted elems
- insertion has good complexity for skip list
- insert = find + actual insertion
- search: $O(\log_2 n)$
- probabilistic DS (height)

SEARCH/CONTAINER
- D.A: $\Theta(n)$
- LL: $\Theta(n)$
- BST: $\Theta(\log n)$
- AVL: $\Theta(\log n)$
- HT: $\Theta(1)$
- BH: $\Theta(n)$ (binary)

DLLANode: (dynamically allocated)
- info: TElem
- next: Integer
- prev: Integer

DLLA
- nodes: DLLANode[]
- cap: Integer
- head: Integer
- tail: Integer
- firstEmpty: Integer
- (size: Integer)

current ← dlla.head
current ← dlla.nodes[current].next

function allocate(dlla) is:

subalgorithm free(dlla, pos) is:

subalgorithm insertPosition(dlla, elem, pos)

DLLAIterator — $\Theta(1)$
- list: DLLA
- currentElement: Integer

subalgorithm init(it, dlla) is:
- it.list ← dlla — $\Theta(1)$
- it.currentElement ← dlla.head

subalgorithm getCurrent(it) is:
- if it.currentElement = -1 then
  - @throw exception — $\Theta(1)$
- getCurrent ← it.list.nodes[it.currentElement].info

subalgorithm next(it)
function valid(it)

Linked lists on Arrays

→ if we work in a programming language with no pointers. (use arrays instead)

	1	2	3	4	5	6	
elems:	46	78	11	6	59	19	...
next:	5	6	1	-1	2	4	

! head = 3

=> order: 11, 46, 59, 78, 19, 6

- insert 44: (3rd position)

	1	2	3	4	5	6	7	8
elems:	~~46~~	78	11	6	59	19		44
next:		6	5	-1	8	4		2

! head = 3

=> order: 11, 59, 44, 78, 19, 6

- linked list of empty positions!

SLLA:
- elems: TElem[] } arrays
- next: Integer[] } arrays
- cap: Integer } indexes
- head: Integer } indexes
- firstEmpty: Integer } indexes

function search(slla, elem) is:
- current ← slla.head ; index of head
- while current ≠ -1 and slla.elems[current] ≠ elem ex
  - current ← slla.next[current]
- end-while ; stop traversal when current = -1
- if current ≠ -1 then
  - search ← True — $O(n)$
- else
  - search ← False
- end-if

end-function

subalgorithm init(slla) is:
- slla.cap ← INIT_CAPACITY
- slla.elems ← @ array w slla.cap pos.
- slla.next ← @ array ...
- slla.head ← -1
- for i ← 1, slla.cap-1 ex
  - slla.next[i] ← i+1
- slla.next[slla.cap] ← -1
- slla.firstEmpty ← 1

subalgorithm insertFirst(slla, elem)

subalgorithm insertPosition(slla, elem, pos)

↓

MERGE SORTED LINKED-LISTS. (SLL)

L_1: [3]→[5]→[9]→[10]→[11|X] (sorted)

L_2: [1]→[2]→[6]→[7|X] (sorted)

L_R: [1]→[2]→[3]→[5]→[6]→[7]→...

- you can just change the links between the nodes if you don't need L_1, L_2.
- otherwise, take L_R (result list)
- Representation:

Node:

value: TComp

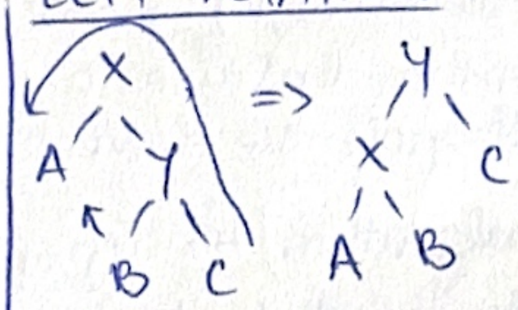
next: ↑Node (pointer)

List:

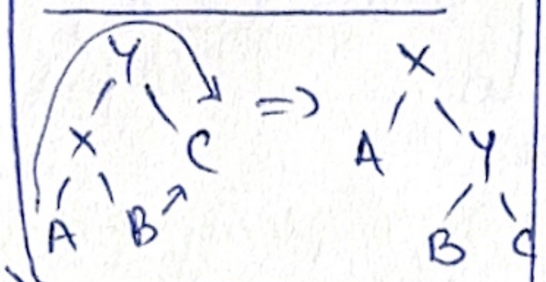
head: ↑Node

(R: Relation)

LEFT ROTATION:



RIGHT ROTATION:



- Implementation:

```
subalgorithm merge(L1, L2, LR):
    if not L1.R([L1.head].value, [L2.head].value) then
        tmp ← L1.head
        L1.head ← L2.head
        L2.head ← tmp
    LR.head ← L1.head (guaranteed the "smaller")
    cL1 ← [L1.head].next (second node of L1)
    cL2 ← L2.head (current first node of L2)
    while cL2 ≠ NiL and cL1 ≠ NiL ex:
        if L1.R([cL1].value, [cL2].value) then
            chosen ← cL1 ; choose a "smaller" (from L1)
            cL1 ← [cL1].next ; advance
        else
            chosen ← cL2
            cL2 ← [cL2].next
        LRtail ← L1.head (initially)
        [LRtail].next ← chosen (attach chosen)
        LRtail ← chosen ; move tail pointer
                          (chosen is after tail)
    if cL1 ≠ NiL ; L1 still has nodes left
        [LRtail].next ← cL1 ; attach directly bc L1
                              is already sorted
    else
        [LRtail].next ← cL2
    LR.R ← L1.R ; copy the ordering relation
    L1.head ← NiL ; empty L1 & L2.
    L2.head ← NiL
```

same relation?

right order?

MOST IMPORTANT DS:

I. DYNAMIC ARRAY (random access)
→ contiguous memory; fast indexing
→ insert/delete is expensive in the middle
- operations: Θ(1): access by index, delete end, insert end Θ(1) amortized;
- op: Θ(n): search, insert/delete middle

II. SLL (many insertions/removals)
→ fast insertion/deletion
→ no positions
→ op: Θ(1): insert/delete front
→ op: Θ(n): search, access pos p

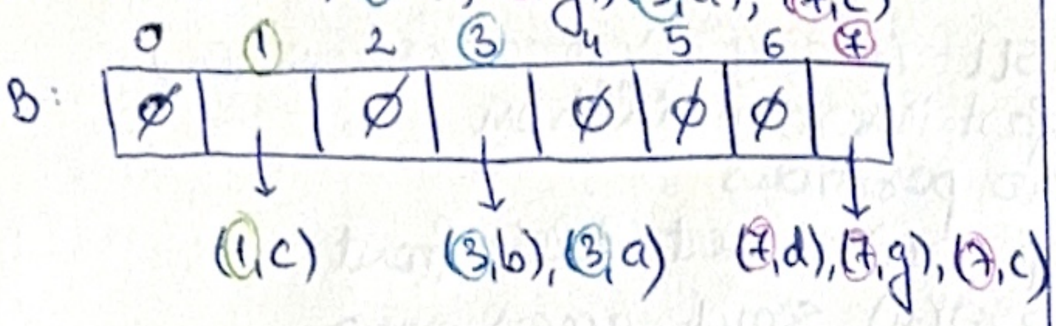
III. DLL
→ fast deletion
→ more memory

IV.

BucketSort (STABLE S.A.)

- Sort a set of pairs based on the keys.
- if we have equal keys, they will appear in the order they were added.

S: (7,d), (1,c), (3,b), (7,g), (3,a), (7,c)



S: (1,c), (3,b), (3,a), (7,d), (7,g), (7,c)

- circular array (DS) if we wanted $\Theta(1)$. (for all operations)

ADT Sequence ~ Queue

$\Theta(1)$
- init(s)
- removeFirst(s)
- addToEnd(s, (k,v))
- destroy(s)
- isEmpty(s)

```
Subalgorithm BucketSort(S,N)
    @define array B of dimension N
    while not isEmpty(S) execute:                    Θ(m)
        (k,v) ← removeFirst(S)
        addToEnd(B[k], (k,v))
    for i=0,N execute:                               Θ(m+N)
        while not isEmpty(B[i]) ex
            (k,v) ← removeFirst(B[i])
            addToEnd(S,(k,v))
```

m = nr of pairs
N = dimension of B
→ keys = integers!

DS Iterator
- data: DS
- current: TElem / ↑Node
- ...
- init
- first
- getCurrent
- valid
- next

LexicographicSort

- Sort a set of d-pairs
- compare the first dimension, if they're equal → the second etc.
- R_i = relation comparing 2 tuples considering the i^{th} position

$(x_1, x_2, x_3, \dots, x_d)$
R_1 R_2 R_d
$(y_1, y_2, y_3, \dots, y_d)$

- stableSort(S,R) - stable sorting algorithm that uses a relation R to compare the elements. (d times)

```
Subalgorithm LexicographicSort(S,d,R)
    for i ← d,1,-1 execute                           Θ(d·T(m))
        stableSort(S,Ri)
```

T(m) = complexity of stableSort

S: (7,4,6), (5,1,5), (2,4,6), (2,1,4), (3,2,4)
d=3: (2,1,4), (3,2,4), (5,1,5), (7,4,6), (2,4,6)
d=2: (2,1,4), (5,1,5), (3,2,4), (7,4,6), (2,4,6)
d=3: (2,1,4), (2,4,6), (3,2,4), (5,1,5), (7,4,6)

RadixSort (STABLE S.A. = BucketSort)

- LexicographicSort + BucketSort

$\Theta(d \cdot (N+m)) \approx \Theta(m)$
- d → nr. of digits in the max. nr.
- N → how many nr. we are sorting
- m → nr. of buckets = nr. digits = 10

(2,4,6) ⇒ 246
(0,3,5) ⇒ 35
You need the same length!

- Linear complexity because it never compares those tuples directly

ADTs

1) ADT Bag → not unique, no pos, no order.
2) ADT SortedBag → no pos, yes order (+relation)
3) ADT Set → ~~not~~ unique, no pos, no order
4) ADT SortedSet → set + relation
5) ADT Matrix → two-dimensional array → sparse matrix if many values of 0 → memorize only ≠0 val.
6) ADT Map → no pos, no order, not necessarily unique values, but keys must be unique,
   ↳ store (key, value) pairs
7) ADT SortedMap → Map with sorted keys (relation)
8) ADT MultiMap → map where a key can have multiple values
9) ADT SortedMultiMap → multimap + ordered keys (values don't matter)
10) ADT Stack → LIFO → access restricted to TOP (NO ITERATOR)
11) ADT Queue → FIFO → push to REAR, pop to FRONT (if we push more) (NO ITERATOR)
12) ADT PriorityQueue → HPF (highest priority first) ⇒ each elem. has an associated priority → NO ITERATOR
13) ADT Deque (double-ended queue) → insert/delete from both ends
14) ADT List → if TPosition = int ⇒ IndexedList
    ↳ if TPosition = iterator ⇒ IteratedList
15) ADT SortedList → indexed/iterated

DFS ↔ STACK

BFS ↔ QUEUE

AVL → if child towards we do the rotation is the same type heavy type as the unbalanced node ⇒ single rot. otherwise ⇒ double rot.